SITE AUTOMATION

All-in-one EE

One execution environment for Linux, Windows and Active Directory, VMware, NetApp, databases, Cisco and ServiceNow

Site Automation - build it, check it, add it to AAP | site-automation 0.29.0 | October 2026

What this is. One execution environment (EE) with everything the site's automation needs: Linux, Windows and Active Directory, VMware, NetApp, MySQL, MariaDB and SQL Server, Cisco, ServiceNow - and every site-automation job. What is in it; how to build it from `ee/all-in-one/`, check it and add it to AAP; Active Directory from it (AD as an inventory, AD on a domain controller, LDAP from the EE); a first task for databases, Cisco and ServiceNow; and what to do when something goes wrong.

How to read it

Steps 1 to 8 in order build it and put it in AAP. The build files themselves are in the appendix. Rendered from `docs/EXECUTION_ENVIRONMENT.md` and `ee/all-in-one/` in the site-automation repository (version 0.29.0).

Host names and addresses in this guide are placeholders. Copy YAML from the repository files, not from the PDF.

Contents

1. The all-in-one execution environment 3

What is in it . . . 4
What you need . . . 5
Step 1. Copy the files . . . 6
Step 2. The base image . . . 6
Step 3. Your automation hub . . . 6
Step 4. Your CA certificates . . . 6
Step 5. Build it . . . 6
Step 6. Check it . . . 7
Step 7. Put it where AAP can pull it . . . 7
Step 8. Add it to AAP . . . 7
Active Directory from the EE . . . 8
Databases, Cisco, ServiceNow: a first task each . . . 9
Updating it . . . 10
When something goes wrong . . . 11

2. The build files (ee/all-in-one/) 12

execution-environment.yml . . . 12
requirements.yml . . . 14
requirements.txt . . . 15
bindep.txt . . . 15
ansible.cfg.example . . . 15
krb5-ad-enctypes.conf . . . 16
check-ee.sh . . . 16

1. The all-in-one execution environment

One execution environment (EE) for everything: Linux, Windows and Active Directory, VMware, NetApp, databases (MySQL, MariaDB, SQL Server), Cisco, ServiceNow - and every site-automation job. You build it once from the files in `ee/all-in-one/`, add it to AAP, and pick it on any template.

Why one
An EE is the container image a job runs in: Ansible, its collections, and the Python libraries and tools they need. A job can only use what its EE has. One EE that has it all means no job fails because "the module is not installed" - and one image to update, check and approve.

What is in it

For	Collections	Python libraries (and tools)	What you can do with it
Linux	`ansible.posix`, `ansible.utils`, `community.general`, `community.crypto`, `containers.podman`, `redhat.rhel_system_roles`	`jmespath`, `netaddr`, `dnspython`, `cryptography`	firewalld, SELinux, mounts, sysctl, podman and Quadlet, certificates, the RHEL System Roles (timesync, crypto policies, logging, SSH ...)
Windows	`ansible.windows`, `community.windows`	`pywinrm` with `pykerberos` (Kerberos) and `requests-ntlm`, `pypsrp`, `requests-kerberos`; `kinit` / `klist`	every Windows module over WinRM; site-automation's Windows jobs
Active Directory	`microsoft.ad`	`sansldap`, `pyspnego` (Kerberos), `krb5`, `dnspython`, `ldap3`	users, groups, computers, OUs, GPOs on a domain controller; **AD as an AAP inventory** (the LDAP inventory plugin); LDAP queries from the EE
VMware	`vmware.vmware`	`pyVmomi`, the vSphere Automation SDK (`vmware-vcenter`); `cryptography` (it comes with ansible-core)	vCenter: VMs, snapshots, clusters, the vCenter and ESXi certificates - site-automation's VMware jobs and the VM deploy
NetApp	`netapp.ontap`	`requests`, `solidfire-sdk-python`, `deepdiff` and the others the collection lists	ONTAP volumes, SVMs, snapshots, SnapMirror (the site-automation ONTAP report needs none of it: it uses Python alone)
Databases	`ansible.mysql` (MySQL and MariaDB), `community.general` (SQL Server: `mssql_db`, `mssql_script`)	`PyMySQL`, `pymssql`	queries, users and databases over the network, from AAP
Cisco and network	`ansible.netcommon`, `cisco.ios`, `cisco.nxos`, `cisco.asa`	`ansible-pylibssh`, `paramiko`, `ncclient`, `textfsm`, `ttp`, `netaddr`, `scp`	IOS / IOS-XE, Nexus and ASA: show commands, configuration, backups, facts
ServiceNow	`servicenow.itsm`	`requests`	incidents, changes - and the **CMDB as an AAP inventory** (`servicenow.itsm.now`)
All	-	your **CA certificates** in its trust store; a Kerberos setting for AD on a **FIPS** host, and Kerberos tickets in a file (the keyring does not work in a container); `ee-check`	TLS to vCenter, ServiceNow, AAP, LDAPS and ONTAP checked against your CA

The collections bring their own Python and system needs (`ansible-builder` reads them from each collection): `requirements.txt` and `bindep.txt` only add what none of them lists.

Tested - and what was not
These files were built on a RHEL 9-compatible base (Rocky Linux 9, `dnf`, Python 3.11 and ansible-core 2.16 installed by the build), without `redhat.rhel_system_roles` (it is only on Automation Hub). `ee-check` said YES to every library and tool, Kerberos and the CA store were as below, the AD inventory file further down was read by its plugin, and all 41 site-automation playbooks passed a syntax check in it. Red Hat's own base image (`microdnf`, its repositories) was not available to the test: your first build is its test - the rows `Python.h` and `No match for argument` in *When something goes wrong* are where a difference would show.

The files (`ee/all-in-one/`):

File	What it holds	You change
`execution-environment.yml`	the recipe: the base image, the three lists below, the build steps	the base image (step 2)
`requirements.yml`	the collections	add or remove one; pin the versions after the first good build
`requirements.txt`	the Python libraries no collection lists	add one
`bindep.txt`	the RHEL packages no collection lists	add one
`ansible.cfg.example`	where the collections come from: your automation hub	copy to `ansible.cfg`, your hub's address (step 3)
`ca/`	your CA certificates, as `.pem` files	put yours in (step 4)
`krb5-ad-enctypes.conf`	Kerberos to Active Directory from a FIPS host	nothing
`check-ee.sh`	`ee-check` inside the image: what it has	nothing

What you need

- **A RHEL 9 machine with podman and ansible-builder** - the AAP server will do. `ansible-builder` comes from the AAP repository (`dnf install ansible-builder`, on a host subscribed to *Ansible Automation Platform*) or from PyPI (`pip install --user ansible-builder`). Rootless podman is fine.
- **The base image:** AAP's minimal EE for your AAP version, from `registry.redhat.io` (`podman login registry.redhat.io` with your Red Hat account or a registry service account) - or the copy already on your AAP server (`podman images | grep ee-minimal`, as the user AAP runs as).
- **The collections:** your automation hub (a Private Automation Hub, or console.redhat.com) and a token for it. The certified collections are in its *rh-certified* repository; `community.general`, `community.crypto`, `community.windows` and `containers.podman` in its *community* repository - if your hub does not sync that one, ask its admin, or remove those four lines.
- **The Python libraries:** pypi.org, or your mirror of it (`PIP_INDEX_URL`, step 5).
- **RHEL packages:** the build machine's RHEL subscription (podman hands it to the build) or your Satellite.

Step 1. Copy the files

Copy the folder `ee/all-in-one/` of the repository to the build machine, e.g. `~/ee-all/`. Work in it.

Step 2. The base image

Open `execution-environment.yml`; the base image is:

```
registry.redhat.io/ansible-automation-platform-27/ee-minimal-rhel9:latest
```

Use **the one for your AAP version**: in AAP, **Automation Execution > Infrastructure > Execution Environments > Minimal execution environment**, its **Image** field. A newer base than your AAP may bring an ansible-core your AAP does not support.

What you should see: `podman pull <that image>` downloads it (or says it is there already).

Step 3. Your automation hub

```shell
cp ansible.cfg.example ansible.cfg
```

In `ansible.cfg`, put your hub's address in the three `url =` lines. AAP shows them: **Automation Content

> **Note**
> Repositories **> a repository >** Copy CLI configuration**. From console.redhat.com instead: one server,

`url = https://console.redhat.com/api/automation-hub/content/published/` with `auth_url = https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-connect/token` - and the four community collections from Galaxy (`url = https://galaxy.ansible.com/`).

The token never goes in the file: it is given at build time (step 5). Get it in AAP: **Automation Content > API token > Load token**.

Step 4. Your CA certificates

Copy your root and intermediate CA certificates into `ca/`, one `.pem` file each (PEM: the text that starts with `-----BEGIN CERTIFICATE-----`). The build adds them to the EE's trust store, and points Python's `requests` at it too (`REQUESTS_CA_BUNDLE`): vCenter, ServiceNow, AAP, LDAPS and ONTAP are then checked against your CA - no `validate_certs: false`. No CA files: the build goes on without.

Step 5. Build it

```shell
read -rs TOKEN          # paste the hub token, then Enter: it stays out of the shell history
ansible-builder create -f execution-environment.yml -c context
podman build --network=host \
  --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN="$TOKEN" \
  --build-arg ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN="$TOKEN" \
  -f context/Containerfile -t site-ee-all:1 context
```

- `ansible-builder create` writes the build (`context/Containerfile`) and does nothing else; `podman build` builds it. **10 to 20 minutes** the first time: it downloads the collections and the Python libraries, and compiles the Kerberos ones.
- `--network=host`: the build uses the machine's own network. Without it, a rootless podman whose network helper is blocked fails with `pasta: permission denied`.
- **A PyPI mirror:** add `--build-arg PIP_INDEX_URL=https://nexus.example.mil/repository/pypi/simple`.
- The token is used only in the step that downloads the collections; it is not in the finished image.
- The tag `:1` is the version: the next build is `:2`, and `:1` stays to go back to.

What you should see: at the end, `Successfully tagged localhost/site-ee-all:1`.

Step 6. Check it

```shell
podman run --rm site-ee-all:1 ee-check
```

It lists the Python version and ansible-core, then `YES` or `no` for every library, the collections with their versions, the tools (`kinit`, `dig` ...), the Kerberos setting and how many of your CA certificates it has. Every line should say `YES`.

Kerberos to Active Directory (from inside the image, with your network):

```shell
podman run --rm -it --network=host site-ee-all:1 kinit svc_aap@EXAMPLE.MIL
```

`Password for svc_aap@EXAMPLE.MIL:` = Kerberos works (the domain in capitals): type the password, then `klist` shows the ticket - or Ctrl+C. `Cannot find KDC for realm` = the EE does not find your domain controllers in DNS; `KDC has no support for encryption type` = the Kerberos setting is missing: see *When something goes wrong*.

Step 7. Put it where AAP can pull it

With a registry (your automation hub has one):

```shell
podman login aap.example.mil
podman tag site-ee-all:1 aap.example.mil/site-ee-all:1
podman push aap.example.mil/site-ee-all:1
```

Without a registry - the image as a file, loaded on every AAP node that runs jobs, as the user AAP runs as:

```shell
podman save -o site-ee-all-1.tar site-ee-all:1     # on the build machine
podman load -i site-ee-all-1.tar                   # on each AAP execution node, as the AAP user
```

Step 8. Add it to AAP

Automation Execution > Infrastructure > Execution Environments > Create execution environment:

- **Name:** `All-in-one EE`
- **Image:** `aap.example.mil/site-ee-all:1` - or, loaded from a file, `localhost/site-ee-all:1`

- **Pull:** *Only pull the image if not present before running* (loaded from a file: *Never pull container before running*)
- **Registry credential:** the one for your hub's registry (none for a loaded image)

Then pick it on a template (**Execution environment**), on an inventory source, or as your organization's default. **Try it first** on one template you know - e.g. *Health check* as Check, or *VM - deploy settings finder*: green as before = the EE works.

Active Directory from the EE

Three ways, from the simplest to set up:

1. AD as an AAP inventory - the LDAP inventory plugin of `microsoft.ad` reads the computer objects and makes each one a host (its name; `ansible_host` = its DNS name), with the groups you choose. It runs in the EE: nothing to install on a domain controller. A file in your project whose name ends with `microsoft.ad.ldap.yml`, e.g. `inventories/site/servers.microsoft.ad.ldap.yml` in your work copy (an update never overwrites `inventories/site/`):

YAML

```yaml
plugin: microsoft.ad.ldap
server: dc01.example.mil          # left out: found from DNS (the domain's SRV records)
tls_mode: ldaps                   # LDAP over TLS, port 636: checked against your CA (step 4)
auth_protocol: simple             # FIPS: simple over LDAPS, or kerberos - never ntlm
search_base: OU=Servers,DC=example,DC=mil
filter: (operatingSystem=*Server*)
attributes:
  operatingSystem:
    os_name:                      # a host variable os_name with the OS as AD has it
  description:
    ad_description:
keyed_groups:
  - key: os_name                  # a group per OS: os_Windows_Server_2022_Standard ...
    prefix: os
```

The account comes from a credential, never the file. **Automation Execution > Infrastructure > Credential Types > Create credential type**, `AD LDAP bind`:

YAML

```yaml
inputs:
  fields:
    - id: username
      type: string
      label: User (user@EXAMPLE.MIL)
    - id: password
      type: string
      label: Password
      secret: true
  required: [username, password]
injectors:
  env:
    MICROSOFT_AD_LDAP_USERNAME: "{{ username }}"
    MICROSOFT_AD_LDAP_PASSWORD: "{{ password }}"
```

Then a credential of that type (a read-only domain account is enough: every domain user can read computer objects), and in your inventory: **Sources > Create source**: *Sourced from a Project*, your project, the file above, the `AD LDAP bind` credential, **Execution environment** `All-in-one EE`, *Overwrite* as you choose.

Sync it. Hosts appear with `os_name`, `ad_description` and `microsoft_ad_distinguished_name`.

2. On a domain controller, over WinRM - the `microsoft.ad` modules run on a Windows host with the Active Directory PowerShell module (a DC, or a management server with RSAT), through the Windows credential you already use. They change AD as well as read it:

```yaml
- name: The computer accounts in the Servers OU
  microsoft.ad.object_info:
    ldap_filter: (objectClass=computer)
    search_base: OU=Servers,DC=example,DC=mil
    properties: [dNSHostName, operatingSystem, lastLogonTimestamp]
  register: ad_computers
  delegate_to: dc01.example.mil
```

Also `microsoft.ad.user`, `group`, `membership`, `computer`, `ou`, `object` - see `ansible-doc microsoft.ad.user`. (site-automation's *Active Directory health report* also runs on a domain controller over WinRM, with PowerShell scripts of its own.)

3. Your own LDAP query in Python - `ldap3` (pure Python) is in the EE for a script of your own.

Check the LDAP side from the EE: `microsoft.ad.debug_ldap_client` (on `localhost`) returns what the EE has for LDAP: its Python packages, its Kerberos, and what DNS says about the domain's LDAP servers.

On a FIPS host: NTLM does not work (no MD4) - use `simple` over LDAPS, or `kerberos` with the user as `user@EXAMPLE.MIL`. Kerberos works because this EE allows AD's AES-SHA1 keys (`krb5-ad-enctypes.conf`).

Databases, Cisco, ServiceNow: a first task each

MySQL / MariaDB (from AAP to port 3306 of the server):

```yaml
- name: The databases
  ansible.mysql.mysql_info:
    login_host: db01.example.mil
    login_user: "{{ db_user }}"
    login_password: "{{ db_password }}"
    filter: [databases, version]
  delegate_to: localhost
```

SQL Server (port 1433, a SQL login):

```yaml
- name: The databases and their state
  community.general.mssql_script:
    login_host: sql01.example.mil
    login_user: "{{ mssql_user }}"
    login_password: "{{ mssql_password }}"
    db: master
    script: SELECT name, state_desc FROM sys.databases
  delegate_to: localhost
```

Windows authentication to SQL Server is not done from the EE: run `Invoke-Sqlcmd` on a Windows host over WinRM instead (`ansible.windows.win_powershell`).

Cisco - the device's group variables (an inventory group `ios`, say):

```yaml
ansible_connection: ansible.netcommon.network_cli
ansible_network_os: cisco.ios.ios
ansible_network_cli_ssh_type: libssh      # ansible-pylibssh; paramiko is the other choice
ansible_become: true
ansible_become_method: enable
```

Then `cisco.ios.ios_facts`, `cisco.ios.ios_command` (`show version`), `cisco.ios.ios_config` - and `cisco.nxos.*`, `cisco.asa.*` the same way (`ansible_network_os: cisco.nxos.nxos`, `cisco.asa.asa`). The credential: a *Machine* credential (user, password, enable password as the privilege escalation password).

ServiceNow CMDB as an inventory - a file ending in `now.yml` with `plugin: servicenow.itsm.now`, and on the inventory source site-automation's *ServiceNow API* credential (it sets `SN_HOST`, `SN_USERNAME`, `SN_PASSWORD`, the names the plugin reads) and this EE. The account needs to read the CMDB (`cmdb_read`).

Updating it

1. Change the files (a collection more, a version pinned).
2. Build with the next tag: `-t site-ee-all:2` (step 5), check it (step 6), push or load it (step 7).
3. In AAP, change the EE's **Image** to `:2`. A problem: change it back to `:1`.

Pin the collections once a build works, so the next build gets the same ones: `podman run --rm site-ee-all:1 ansible-galaxy collection list` gives each version; write it in `requirements.yml` (`version: "==3.8.0"`).

When something goes wrong

You see	It means	Do
`unauthorized` / `invalid username/password` pulling `registry.redhat.io/...`	podman is not logged in to Red Hat's registry	`podman login registry.redhat.io`; or use the base image already on your AAP server (step 2)
`pasta ... permission denied` / `netavark` errors	rootless podman's network helper is blocked	`--network=host` on `podman build` (step 5)
`HTTP Code: 401` or `403` installing collections	no token, or the wrong one	the `--build-arg ..._TOKEN` lines; a new token: *Automation Content* > *API token*
`Failed to find collection community.general` (or another)	your hub does not sync that repository	ask the hub's admin to sync it - or remove that line from `requirements.yml`
`No match for argument: <package>`	that RHEL package is in a repository the build does not have (e.g. CodeReady Builder)	when a collection asks for it only to compile: add it under `exclude: system:` in `execution-environment.yml`
`Python.h: No such file or directory`	the C headers of the EE's Python are missing	the build installs `python3.X-devel` for the EE's own Python: check that the base image's repositories have it
`Could not find a version that satisfies the requirement ...`	the build cannot reach PyPI	`--build-arg PIP_INDEX_URL=...` (your mirror)
`ee-check` says `no` for a library	a step of the build was skipped, or the library is not for this Python	read the build's output for that library
a job: `certificate verify failed`	the EE does not have your CA	put it in `ca/` (step 4), build `:2`
a job: `ntlm: ... unsupported`	NTLM on a FIPS host	Kerberos: the user as `user@EXAMPLE.MIL`
`kinit: Cannot find KDC for realm "EXAMPLE.MIL"`	the EE does not find the domain's Kerberos servers in DNS	`podman run ... --network=host`; check `dig _kerberos._tcp.example.mil SRV` inside the EE: it must list your domain controllers
a job: `KDC has no support for encryption type`	the Kerberos setting is not in the image	`ee-check`: *crypto-policies link removed* and the `permitted_enctypes` line - rebuild from these files
a job: `couldn't resolve module/action`	the collection is not in this EE, or the template uses another EE	the template's **Execution environment**; `ee-check` lists the collections

2. The build files (ee/all-in-one/)

The files as they are in the repository, folder `ee/all-in-one/`. Copy them from the repository (or the release zip) where you can: PDF copy can change spaces.

execution-environment.yml

execution-environment.yml

```
---
# The all-in-one execution environment: Linux, Windows and Active Directory, VMware, NetApp,
  databases
# (MySQL, MariaDB, SQL Server), Cisco, ServiceNow - and every site-automation job.
# How to build it, check it and add it to AAP: docs/EXECUTION_ENVIRONMENT.md (also a PDF).
#   cp ansible.cfg.example ansible.cfg            # your automation hub's address
#   ansible-builder create -f execution-environment.yml -c context
#   podman build --network=host --build-arg ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN=... \
#     -f context/Containerfile -t site-ee-all:1 context
version: 3

images:
  base_image:
    # The same base as your AAP's own EEs, for your AAP version (Automation Execution >
      Infrastructure >
    # Execution Environments > Minimal execution environment > Image).
    name: registry.redhat.io/ansible-automation-platform-27/ee-minimal-rhel9:latest

dependencies:
  galaxy: requirements.yml
  python: requirements.txt
  system: bindep.txt
  exclude:
    system:
      # ansible.netcommon asks for these to compile ansible-pylibssh, which comes ready-built (a
        wheel);
      # both are in the CodeReady Builder repository, which a RHEL build does not have
      - python3-Cython
      - libssh-devel

options:
  package_manager_path: /usr/bin/microdnf

additional_build_files:
  - src: ansible.cfg
    dest: configs
  - src: krb5-ad-enctypes.conf
    dest: configs
  - src: check-ee.sh
    dest: configs
  - src: ca
    dest: ca

additional_build_steps:
  prepend_galaxy:
    - COPY _build/configs/ansible.cfg /etc/ansible/ansible.cfg
    # the automation hub token, given at build time - never written in a file
    - ARG ANSIBLE_GALAXY_SERVER_RH_CERTIFIED_TOKEN
```

execution-environment.yml (continued)

```
    - ARG ANSIBLE_GALAXY_SERVER_COMMUNITY_TOKEN
    - ARG ANSIBLE_GALAXY_SERVER_PUBLISHED_TOKEN
  prepend_builder:
    # a PyPI mirror, when the build host cannot reach pypi.org: --build-arg
      PIP_INDEX_URL=https://...
    - ARG PIP_INDEX_URL
    # the C headers of the EE's own Python (gssapi, krb5 and pykerberos compile against them), and
      pip's
    # wheel package: pykerberos (WinRM's Kerberos) is then built here, not in the image, which has
      no compiler
    - RUN $PKGMGR install -y python$($PYCMD -c 'import sys; print("%d.%d" %
      sys.version_info[:2])')-devel && $PKGMGR clean all
    - RUN $PYCMD -m pip install --no-cache-dir wheel
  append_final:
    # Kerberos to Active Directory from a FIPS host: AD's AES-SHA1 keys allowed
      (krb5-ad-enctypes.conf)
    - COPY _build/configs/krb5-ad-enctypes.conf /etc/krb5.conf.d/ad-enctypes.conf
    - RUN rm -f /etc/krb5.conf.d/crypto-policies
    # Kerberos tickets in a file: the kernel keyring RHEL uses by default does not work in a
      container
    - ENV KRB5CCNAME=FILE:/tmp/krb5cc_ee
    # your CA certificates (ca/*.pem) in the trust store - vCenter, ServiceNow, AAP, LDAPS, ONTAP
    - COPY _build/ca/ /tmp/site-ca/
    - RUN cp /tmp/site-ca/*.pem /etc/pki/ca-trust/source/anchors/ 2>/dev/null; update-ca-trust
      extract && rm -rf /tmp/site-ca
    - ENV REQUESTS_CA_BUNDLE=/etc/pki/tls/certs/ca-bundle.crt
    # ee-check: what the image has (podman run --rm <image> ee-check)
    - COPY _build/configs/check-ee.sh /usr/local/bin/ee-check
    - RUN chmod 755 /usr/local/bin/ee-check
```

requirements.yml

requirements.yml

```
---
# The Ansible collections of the all-in-one execution environment (ee/all-in-one,
  docs/EXECUTION_ENVIRONMENT.md).
# "certified" = Red Hat Automation Hub (also in a Private Automation Hub that syncs it);
  "community" = Galaxy
# (a Private Automation Hub needs its community repository synced for these). ansible-builder
  reads each
# collection's own Python and system needs: they are not repeated in requirements.txt /
  bindep.txt.
# After the first good build, pin each one to the version it got (ansible-galaxy collection list).
collections:
  # ---- Linux ----
  - name: ansible.posix                # certified: acl, firewalld, mount, sysctl, selinux ...
  - name: ansible.utils                # certified: ipaddr filters, validate, cli_parse
  - name: community.general            # community: hundreds of modules (ldap_*, mssql_*, nmcli,
    ...)
  - name: community.crypto             # community: certificates, CSRs, keys
  - name: containers.podman            # community: podman containers, pods, Quadlet
  - name: redhat.rhel_system_roles     # certified (Automation Hub only): RHEL System Roles
  # ---- Windows and Active Directory ----
  - name: ansible.windows              # certified: the core Windows modules
  - name: microsoft.ad                 # certified: AD objects, users, groups - and the LDAP
    inventory plugin
  - name: community.windows            # community: more Windows modules
  # ---- VMware ----
  - name: vmware.vmware                # certified: vCenter (site-automation's VMware jobs need
    >= 2.11)
    version: ">=2.11.0"
  # ---- NetApp ----
  - name: netapp.ontap                 # certified: ONTAP REST modules
  # ---- Databases ----
  - name: ansible.mysql                # certified: MySQL and MariaDB (was community.mysql)
  # ---- Cisco and network ----
  - name: ansible.netcommon            # certified: network_cli, netconf, httpapi connections
  - name: cisco.ios                    # certified: IOS / IOS-XE
  - name: cisco.nxos                   # certified: Nexus
  - name: cisco.asa                    # certified: ASA firewalls
  # ---- ServiceNow ----
  - name: servicenow.itsm              # certified: tickets, and the CMDB inventory plugin
    (servicenow.itsm.now)
```

requirements.txt

requirements.txt

```
# Python libraries the collections do not bring themselves (ansible-builder adds theirs).
# Windows over WinRM (ansible.windows, community.windows, microsoft.ad modules, site-automation's
  Windows jobs)
pywinrm[kerberos]
requests-kerberos
requests-ntlm          # NTLM: not on a FIPS host (no MD4); Kerberos is the way there
pypsrp[kerberos]       # the psrp connection, an alternative to winrm
# Active Directory from the EE itself, over LDAP (microsoft.ad brings sansldap, pyspnego,
  dnspython)
ldap3                  # pure Python: your own scripts, no compiler
# SQL Server (community.general.mssql_db, mssql_script)
pymssql
# Linux and general
jmespath               # the json_query filter
dnspython              # the community.general.dig lookup
```

bindep.txt

bindep.txt

```
# System packages (RPMs) the collections do not bring themselves (ansible-builder adds theirs).
# [compile] = only while building; the rest stays in the image.
krb5-workstation [platform:rpm]       # kinit / klist: check Kerberos inside the EE
openssh-clients [platform:rpm]
sshpass [platform:rpm]                # password SSH (network devices that need it)
iputils [platform:rpm]                # ping, for troubleshooting from the EE
bind-utils [platform:rpm]             # dig / nslookup: check DNS (the domain controllers' SRV
  records)
```

ansible.cfg.example

ansible.cfg.example

```
# Copy to ansible.cfg (this folder) when the collections come from your automation hub. The URLs:
  AAP >
# Automation Content > Repositories > a repository > "Copy CLI configuration" (AAP 2.5 and later:
  the
# platform's own address). The token is NOT written here: it is given at build time (--build-arg).
[galaxy]
server_list = rh_certified, community, published

[galaxy_server.rh_certified]
url = https://aap.example.mil/api/galaxy/content/rh-certified/

[galaxy_server.community]
url = https://aap.example.mil/api/galaxy/content/community/

[galaxy_server.published]
url = https://aap.example.mil/api/galaxy/content/published/
```

krb5-ad-enctypes.conf

krb5-ad-enctypes.conf

```
# Kerberos to Active Directory from a FIPS host (STIG). The EE's Kerberos would follow the host's
  FIPS
# policy and offer only AES with SHA-2, while Active Directory's keys are AES with SHA-1: "KDC has
  no
# support for encryption type". This allows both AES kinds - never RC4 or DES.
[libdefaults]
  permitted_enctypes = aes256-sha2 aes128-sha2 aes256-sha1 aes128-sha1
```

check-ee.sh

check-ee.sh

```
#!/bin/bash
# ee-check: what this execution environment has. Run it inside the image:
#   podman run --rm <image> ee-check
# the Python Ansible runs on (the first line of the ansible command), not the system's python3
py=$(head -1 "$(command -v ansible)" 2>/dev/null | sed -e 's/^#! *//' -e 's/ .*//')
[ -x "$py" ] || py=python3
echo "== Python: $($py --version 2>&1)   ansible-core: $(ansible --version 2>/dev/null | head -1)"
echo "== Python libraries"
for m in winrm:pywinrm requests_kerberos:requests-kerberos requests_ntlm:requests-ntlm
  pypsrp:pypsrp \
         spnego:pyspnego gssapi:gssapi krb5:krb5 sansldap:sansldap ldap3:ldap3 dns:dnspython \
         pyVmomi:pyVmomi com.vmware.vcenter_client:vmware-vcenter \
         kerberos:pykerberos requests:requests solidfire:solidfire-sdk-python \
         pymysql:PyMySQL pymssql:pymssql \
         pylibsshext:ansible-pylibssh paramiko:paramiko ncclient:ncclient netaddr:netaddr
           textfsm:textfsm \
         jmespath:jmespath cryptography:cryptography; do
  mod=${m%%:*}; name=${m#*:}
  if $py -c "import $mod" 2>/dev/null; then echo "  YES  $name"; else echo "  no   $name"; fi
done
echo "== Collections"
ansible-galaxy collection list 2>/dev/null | grep -E '^[a-z0-9_]+\.[a-z0-9_]+ ' | sort
echo "== Tools"
for t in kinit klist ssh sshpass dig ping; do
  if command -v $t >/dev/null; then echo "  YES  $t"; else echo "  no   $t"; fi
done
echo "== Kerberos (FIPS, Active Directory)"
if [ -e /etc/krb5.conf.d/crypto-policies ]; then echo "  /etc/krb5.conf.d/crypto-policies is
  there: on a FIPS host AD's AES-SHA1 keys are refused"
else echo "  crypto-policies link removed"; fi
grep -h permitted_enctypes /etc/krb5.conf.d/* 2>/dev/null | sed 's/^ */  /'
echo "== Your CA certificates added: $(ls /etc/pki/ca-trust/source/anchors/ 2>/dev/null | wc -l)"
```